

ThreatAttest — Getting Started

ThreatAttest — Getting Started

This guide walks through installing and running **ThreatAttest**: a Cosmos SDK blockchain for decentralized threat intelligence, plus the **ThreatAttest Shield** browser extension that consumes it for real-time protection.

It covers a from-scratch setup on Linux/macOS (and Windows via WSL/Docker).

1. Overview

ThreatAttest has two components:

Component	Directory	Language	What it does
Chain node	chain/	Go	<code>threatattestd</code> — publishes/endororses/disputes/revokes threat attestations
Browser extension	extension/	JavaScript	ThreatAttest Shield — checks downloads/URLs/domains/IPs against the chain

The chain exposes three APIs:

API	Port
REST (LCD)	1317
gRPC	9090
CometBFT RPC	26657

The extension is a **read-only** client of the REST API — it never holds keys or signs transactions.

2. Prerequisites

Tool	Version	Check
Go	≥ 1.23	<code>go version</code>
Node.js	≥ 20	<code>node --version</code>
make	any	<code>make --version</code>
Docker	≥ 24 <i>(optional — for Docker deployment)</i>	<code>docker --version</code>
git	any	<code>git --version</code>

3. Clone and install

```
git clone <repository-url> threatattest
cd threatattest
```

```
# Verify the toolchain and show all available build targets
make help
```

The root Makefile orchestrates both sub-projects. Key targets:

Target	Purpose
<code>make build</code>	Build chain binary + extension
<code>make build-chain</code>	Build threatattestd only
<code>make build-ext</code>	Build the browser extension only
<code>make test</code>	Run all tests (Go + shell + JS)
<code>make localnet</code>	Init + start a single-node devnet
<code>make docker-build</code>	Build the Docker image

4. Build the chain

```
# From the repo root
make build-chain
# → produces chain/bin/threatattestd
```

Verify it works:

```
./chain/bin/threatattestd version
```

To install the binary onto `$PATH` (`$GOPATH/bin`):

```
make install
```

5. Run the tests *(optional but recommended)*

```
make test          # everything: Go unit, shell integration,
                   JS extension
make test-chain    # Go unit tests only
make test-shell    # offline shell integration tests (no node
                   required)
make test-js       # extension JS tests (151 badge/identity
                   tests)
```

The shell tests run without a node; integration sections are skipped gracefully when no node is up.

6. Start a local devnet

The `localnet` targets script a complete single-node devnet.

```
make localnet      # = localnet-init + localnet-start
```

Defaults:

Setting	Value
Binary	chain/bin/threatattestd
Home dir	~/.threatattestd-local
Chain ID	threatattest-local-1
Key name	localkey
Denom	utatst
Moniker	localnode
Initial balance	200,000,000 utatst

Manage the node:

```
make localnet-status  # query node status
make localnet-log     # tail the node log
make localnet-stop    # stop the node
make localnet-reset   # wipe and re-init
```

You can override the defaults with `init-localnet.sh` options:

```
cd chain
./scripts/init-localnet.sh --home ~/my-node --chain my-test-1
--key alice
```

7. Interact with the chain (CLI)

With the node running, the binary is at `chain/bin/threatattestd`. Use `--home ~/.threatattestd-local` (and `--keyring-backend test`) to reach the devnet.

Convenience helpers that build attestation JSON specs:

```
threatattestd attest-file <file> # hash a file →
    attestation spec
threatattestd attest-url <url> # normalize URL →
    attestation spec
threatattestd attest-domain <domain> # normalize domain →
    attestation spec
```

Attestation transactions:

```
threatattestd tx attestation publish ... # publish a new
    attestation
threatattestd tx attestation endorse <id> # endorse an
    attestation
threatattestd tx attestation dispute <id> # file a dispute
threatattestd tx attestation revoke <id> # revoke your own
    attestation
threatattestd tx attestation claim-reward
    # claim a share of the incentive pool
threatattestd tx attestation subscribe ... # lock TATST →
    paid API tier
threatattestd tx attestation unsubscribe # return locked
    stake → FREE tier
```

Identity transactions:

```
threatattestd tx identity register-identity ... # register
    DNS-anchored identity
threatattestd tx identity rotate-identity-key ... # rotate
    signing key
threatattestd tx identity renew-identity ...
    # renew with fresh DNSSEC evidence
threatattestd tx identity revoke-identity ... # revoke
    identity
```

Run any command with `--help` for the full flag set.

8. Install the browser extension (ThreatAttest Shield)

```
# From the repo root
make build-ext          # production build → extension/dist/
make build-ext-dev      # development build (points at localhost API)
make pack-ext           # package into extension/build/*.zip
```

Load it into Chrome/Edge/Brave:

1. Open `chrome://extensions`.
2. Enable **Developer mode** (top-right toggle).
3. Click **Load unpacked**.
4. Select the `extension/dist/` directory.
5. Open the extension popup and set the node URL — for the local devnet use `http://localhost:1317`.

The dev build (`make build-ext-dev`) already points at `localhost:1317`. The production build defaults to `https://api.threatattest.io`.

The extension checks every completed download (SHA-256), every navigated URL, and page IPv4 addresses against the chain, blocking CRITICAL/HIGH threats and showing DNS-identity badges for verified domains.

9. Bind a DNS identity (*optional but recommended*)

To publish attestations with a *DNS-bound* or *Expert* tier (higher confidence multiplier), bind a domain to your Cosmos key with a TXT record:

```
_tat.<selector>.<domain>. 3600 IN TXT "v=TAT1 k=secp256k1
p=<pubkey-hex> a=<cosmos-addr>"
```

- `<selector>` — a short DNS label (e.g. `validator`).
- `<pubkey-hex>` — the compressed secp256k1 public key (hex).
- `<cosmos-addr>` — your `tatst1...` account address.

The localnet init script prints a ready-to-use record after creating the validator key. After publishing the record, register it on-chain:

```
threatattd tx identity register-identity ... --public-key-  
hex <hex> --domain <domain> --selector <selector>
```

Full identity/DNSSEC details are in [system-requirements.md](#).

10. Docker (*alternative to the local devnet*)

```
make docker-build           # build the image  
make docker-run             # single-node container  
make docker-compose-up COMPOSE_PROFILES=monitoring # node +  
                           Prometheus + Grafana  
make docker-compose-down
```

The compose stack maps the same ports (26656/26657/1317/9090) and supports `CHAIN_ID`, `DENOM`, `SUPPLY_AMOUNT`, etc. as environment variables.

11. Production build & release

```
make release-snapshot # build all release artifacts locally  
                      (no publish)
```

Release artifacts (via GoReleaser, on `v*.*.*` tags) include multi-arch `threatattd` binaries, checksums, Docker manifests, and the packaged extension zip — see [system-requirements.md](#) §8 and [.github/workflows/release.yml](#).

12. Troubleshooting

Symptom	Fix
<code>make build-chain</code> fails on a Go version error	Install Go ≥ 1.23 and re-run
<code>make localnet-start</code> reports the node is already running	<code>make localnet-reset</code>
Extension shows “node unreachable”	Confirm the node is up and the URL is <code>http://localhost:1317</code>
<code>tx ...</code> says “key not found”	Use <code>--home ~/.threatattd-local --keyring-backend test</code>

Symptom	Fix
Port already in use	Change <code>*_PORT</code> in <code>docker-compose.yml</code> , or stop the localnet

13. Reference documents

The reference documents linked throughout this guide are included in full below.

ThreatAttest — System Requirements

Decentralized threat intelligence on a Cosmos SDK blockchain with a browser extension for real-time protection.

1. System Overview

ThreatAttest is a two-component system:

Component	Description	Language	Build Output
chain/	Cosmos SDK blockchain node (<code>threatattestd</code>)	Go 1.23	Single binary
extension/	ThreatAttest Shield browser extension (MV3)	JavaScript	.zip for Chrome Web Store

The chain publishes, endorses, disputes, and revokes cryptographically verifiable threat intelligence attestations. The extension consumes the chain's REST API to provide real-time protection in the browser — checking downloads, URLs, domains, and IPs against on-chain attestations.

2. Functional Requirements

2.1 Chain — Attestation Module (x/attestation)

FR-ATT-01 — The system shall accept attestation publications for artifacts of type FILE, URL, IPV4, and DOMAIN.

FR-ATT-02 — Each attestation shall be uniquely identified by `SHA-256(artifact_sha256 | attester | published_at)`.

FR-ATT-03 — Attestations shall carry: artifact SHA-256, severity (INFO/LOW/MEDIUM/HIGH/CRITICAL), TLP level, confidence (0–100), TTL, description, tags, threat categories (17 TATST-prefixed types), detection rules (up to 32), MITRE ATT&CK IDs, and PUA metadata.

FR-ATT-04 — Attesters shall be rate-limited per epoch (default: 100 attestations/epoch, ~1 hour).

FR-ATT-05 — Attester confidence shall be scaled by identity compliance tier: - Anonymous: 0.25× - Staked: 0.50× - DNS_Bound: 1.00× - Expert: 2.00×

FR-ATT-06 — The system shall support endorsement of attestations by other attesters, incrementing endorsement count and trust score.

FR-ATT-07 — The system shall support disputes on five grounds: FALSE_POSITIVE, INCORRECT_SEVERITY, FABRICATED_EVIDENCE, STALE_REUSE, SYBIL_ATTACK.

FR-ATT-08 — Only the original attester may revoke their own attestation.

FR-ATT-09 — Attestations shall auto-expire after their TTL elapses (EndBlocker sweep of expiry queue).

FR-ATT-10 — The system shall provide queries: is-malicious (by SHA-256, URL, or IPv4), get-attestation (by ID), list-by-artifact, list-by-attester, get-dispute, is-blacklisted, params.

FR-ATT-11 — Trust score shall be computed as:

```
severity_weight × (confidence/100) × (1 + log2(1 +  
endorsements)) × tier_weight
```

FR-ATT-12 — Attesters shall claim a share of the attestation incentive pool via `MsgClaimReward`. Each claim withdraws 1% of the pool balance, is rate-limited to one claim per attester per epoch, and requires a non-empty pool.

FR-ATT-13 — The system shall support paid API subscription tiers via `MsgSubscribe/MsgUnsubscribe`:

Tier	Stake	Lock	Rate limit	Features
FREE	0	—	10 req/min	SHA-256 lookup
PROFESSIONAL	1,000 TATST	90 days	1,000 req/min	all
ENTERPRISE	100,000 TATST	365 days	10,000 req/min	all

Re-subscribing refunds the prior stake before locking the new one; `MsgUnsubscribe` returns the full stake.

2.2 Chain — Identity Module (`x/identity`)

FR-ID-01 — The system shall bind Cosmos addresses to DNSSEC-secured domain names via DNS TXT records.

FR-ID-02 — Identity registration shall require: domain proof signature, DNSSEC evidence bundle (TXT records, RRSIGs, DNSKEYs, DS records), and optional flags (EXPERT, ORGANIZATION, GOVERNMENT, SECURITY_FIRM, AUTOMATED, MULTISIG).

FR-ID-03 — The system shall support four compliance tiers affecting attestation confidence: ANONYMOUS, STAKED, DNS_BOUND, EXPERT.

FR-ID-04 — Identity key rotation shall require dual signatures (old key + new key), following DKIM-style dual-selector rotation.

FR-ID-05 — Identities shall expire after 1 year (default) and require renewal with fresh DNSSEC evidence.

FR-ID-06 — Revoked identities shall incur a -100 reputation score penalty and be permanent.

FR-ID-07 — The system shall validate DNSSEC evidence: freshness (1h max), algorithm allow-list (ECDSA P-256 SHA-256/algorithm 13 as RECOMMENDED), TXT record format (`v=TAT1 k=secp256k1 p={hex} a={addr}`).

2.3 Chain — Reputation Module (x/reputation)

FR-REP-01 — The system shall track non-transferable reputation scores per attester address (0–10,000).

FR-REP-02 — Scores shall decay at 10 basis points/day after 30 days of inactivity (lazy decay on read/write).

FR-REP-03 — Lifecycle events shall adjust scores: registration +200, 30-day milestone +50, renewal +10, DNS removed –200, revoked –100.

FR-REP-04 — The system shall support blacklisting of attesters via governance.

2.4 Chain — IPFS Verify Module (x/ipfsverify)

FR-IPFS-01 — The system shall verify that detection rules (YARA, Sigma, Snort, Suricata, OpenIOC) stored on IPFS match their claimed SHA-256 hashes.

FR-IPFS-02 — An off-chain verifier daemon shall submit verification reports. Verification jobs are created on-chain and enqueued; reports are submitted by whitelisted relayers.

FR-IPFS-03 — Verification jobs shall timeout after 300 blocks (~10 minutes, configurable), with up to 3 retries.

2.5 Chain — Token Minting (x/tatmint)

FR-MINT-01 — The system shall mint TATST tokens with a halving schedule: initial 10 TATST/block, halving every 2,102,400 blocks (~4 years), 30 halvings max, 1 billion TATST supply cap.

FR-MINT-02 — Distribution split: 50% validators, 20% community pool, 30% attester incentives.

2.6 Browser Extension

FR-EXT-01 — The extension shall check every completed download against the chain's `is-malicious` endpoint by SHA-256.

FR-EXT-02 — The extension shall check every navigated URL and domain against the chain's `is-malicious-url` and `is-malicious` (SHA-256 of domain) endpoints.

FR-EXT-03 — The extension shall extract IPv4 addresses from page resources (meta tags, script/img/iframe/link URLs, CSP, dns-prefetch, performance entries) and check them against the chain's `is-malicious-ipv4` endpoint.

FR-EXT-04 — On threat detection, the extension shall: - CRITICAL/HIGH: auto-block the action (cancel download, block navigation) - MEDIUM/LOW/INFO: display an inline overlay banner with Block/Allow/Detail buttons - All severities: update the toolbar badge (green = clean, yellow = MEDIUM, red = HIGH/CRITICAL)

FR-EXT-05 — Download threats shall present a standalone decision popup with Block & Delete, Allow Once, Snooze 30 min, and Always Allow options.

FR-EXT-06 — The extension shall display DNS identity badges for domains with on-chain identity records, showing tier, trust score, and flags.

FR-EXT-07 — API responses shall be cached in-memory with a 5-minute TTL (2-minute for negative identity lookups).

FR-EXT-08 — The extension shall perform periodic node health checks every 2 minutes via the `/cosmos/base/tendermint/v1beta1/node_info` endpoint.

FR-EXT-09 — Files over 50 MB shall be skipped. Files from URLs that cannot be re-fetched (HTTP auth) shall be skipped.

FR-EXT-10 — The extension shall support configurable node URL, severity thresholds, auto-block rules, notifications, and domain/IP whitelists.

3. Non-Functional Requirements

3.1 Chain

Requirement	Value
Consensus	CometBFT 0.38.10, 2s blocks
Bech32 prefix	tatst
Token denom	utatst (micro), tatst (whole)
State store	

Requirement	Value
	Cosmos SDK KV store (GoLevelDB default)
Validators	100 max, 21-day unbonding
Governance	10,000 TATST min deposit, 2-day voting, 33.4% quorum
Slashing	5% double-sign, 1% downtime, 100-block signed window
gRPC	Port 9090
REST (LCD)	Port 1317
RPC	Port 26657
P2P	Port 26656
Min attestation TTL	3,600 seconds (1 hour)
Max attestation TTL	31,536,000 seconds (1 year)
Max IPv4 TTL	7,776,000 seconds (90 days)
Max detection rules	32 per attestation
Identity TTL	31,536,000 seconds (1 year default)
DNS verification epoch	43,200 blocks (~24 hours)
Expert tier threshold	5,000 reputation score
Block reward halving	Every 2,102,400 blocks (~4 years)

3.2 Extension

Requirement	Value
Manifest version	MV3
Browser support	Chromium 88+ (Chrome, Edge, Brave)
Default chain endpoint	http://localhost:1317 (dev) / https://api.threatattest.io (prod)
API cache TTL	5 minutes (positive), 2 minutes (negative identity)
Max download size	50 MB
Max threats per tab	20
Node health interval	2 minutes
Identity badge lookup	On navigation + on popup open
Permissions	downloads, webRequest, storage, notifications, tabs, activeTab, scripting, alarms, declarativeNetRequest
Host access	<all_urls>

3.3 Security

Requirement	Value
Identity proof	DNSSEC chain-of-trust (ECDSA P-256 SHA-256 / algorithm 13)
Attestation signing	secp256k1 (Cosmos key)
Artifact hashing	SHA-256
Attestation ID	SHA-256(artifact_sha256)
Identity ID	SHA-256(domain)
Dispute bond	500 TATST
Blacklist	Governance-gated
Confidence scaling	By identity tier (0.25× - 2.00×)
IPFS rule verification	Off-chain verifier daemon, whitelisted relayers
Extension content script	Isolated world (MV3 default)
API communication	HTTP (localhost dev) / HTTPS (production)

4. API Endpoints (Chain REST)

4.1 Attestation Queries

Method	Path	Description
GET	/threatattest/attestation/v1/is-malicious?artifact_sha256={hex}	Check SHA-256
GET	/threatattest/attestation/v1/is-malicious-url?url={url}	Check URL
GET	/threatattest/attestation/v1/is-malicious-ipv4?ipv4={ip}	Check IPv4
GET	/threatattest/attestation/v1/attestation/{id}	Get attestation
GET	/threatattest/attestation/v1/list-by-artifact?artifact_sha256={hex}	List by SHA-256
GET	/threatattest/attestation/v1/list-	List by attester

Method	Path	Description
	by-attester? attester={addr}	

4.2 Identity Queries

Method	Path	Description
GET	/identity/address/{addr}	Get identity by address
GET	/identity/domain/{domain}	Get identity by domain
GET	/identity/id/{id}	Get identity by ID

4.3 Node Status

Method	Path	Description
GET	/cosmos/base/ tendermint/ v1beta1/node_info	Node health + version

5. CLI Commands (threatattestd)

5.1 Convenience Commands

Command	Description
attest-file <file>	Hash a file and produce attestation JSON spec
attest-url <url>	Normalize URL and produce URL attestation spec
attest-domain <domain>	Normalize domain and produce domain attestation spec

5.2 Attestation Transactions

Command	Description
tx attestation publish	Publish a new attestation
tx attestation endorse <id>	Endorse an existing attestation
tx attestation revoke <id>	Revoke your attestation
tx attestation dispute <id>	File a dispute

5.3 Identity Transactions

Command	Description
tx identity register-identity	Register DNS-anchored identity
tx identity rotate-identity-key	Rotate signing key
tx identity revoke-identity	Revoke identity
tx identity renew-identity	Renew with fresh DNSSEC evidence

6. Data Flow



7. Deployment

7.1 Chain (Production)

```
# Build
cd chain && make build
```

```
# Init
./bin/threatattd init my-node --chain-id threatattd-1

# Start
./bin/threatattd start --minimum-gas-prices=0.001utast

# Docker
cd chain && docker compose up -d
```

7.2 Chain (Local Development)

```
cd chain && make localnet-init localnet-start
```

7.3 Extension (Development)

```
cd extension && make build-dev
# Load unpacked from extension/dist/ in chrome://extensions
```

7.4 Extension (Production)

```
cd extension && make pack
# Upload build/threatattd-shield-<version>.zip to Chrome Web
Store
```

8. External Dependencies

Dependency	Purpose	Version/Notes
Cosmos SDK	Blockchain framework	v0.50.9
CometBFT	Consensus engine	v0.38.10
GoLevelDB	State storage	Via cosmos-db
IPFS	Detection rule storage	Off-chain; gateway URL configurable
DNSSEC	Identity verification	TXT records, algorithm 13 (ECDSA P-256)
Prometheus	Metrics	Optional, via docker-compose profile
Grafana	Dashboards	Optional, via docker-compose profile
Chrome Extensions API	Browser integration	MV3, Chromium 88+

9. Constraints

1. **No external databases** — All chain state in Cosmos SDK KV store. No PostgreSQL, MySQL, or Redis.
 2. **No direct blockchain access from extension** — Extension is read-only REST API consumer.
 3. **No key material in extension** — Extension holds no private keys; cannot sign or transact.
 4. **IPFS is off-chain** — Chain stores CIDs and verification results; content fetching is delegated.
 5. **DNSSEC chain-of-trust is stub** — Cryptographic signature verification is structural; full chain validation pending production DNS library.
 6. **Single-node in docker-compose** — Production deployment needs validator set configuration.
 7. **50 MB download limit** — Extension skips files larger than 50 MB to avoid memory issues.
-

ThreatAttest Bootstrapping Guide

Overview

A new ThreatAttest chain starts with zero tokens. This guide covers how tokens enter circulation and how attesters obtain gas.

Token Flow

Block reward (10 TATST/block)

- └─ 50% → validators/delegators (staking rewards)
- └─ 20% → community pool (governance controlled)
- └─ 30% → attestation module pool (claimable by attesters)

Bootstrapping Steps

1. Initialize chain with funded validator

```
# Build the binary
cd chain && make build

# Run localnet initialization
bash scripts/init-localnet.sh
```

This creates: - A validator key (localkey) in the test keyring - A genesis account for the validator with 200 TATST - A gentx self-delegation of 100 TATST - Patched genesis.json with sensible attestation parameters - Enabled REST API, gRPC, and CORS

2. Start the chain

```
bash scripts/start-localnet.sh
```

The validator begins producing blocks immediately. Block rewards start flowing on block 1: - 5 TATST to the validator (via fee_collector → distribution) - 2 TATST to the community pool - 3 TATST to the attestation incentive pool

3. Verify the chain is running

```
# Check block height
curl -s http://localhost:26657/status | jq
    '.result.sync_info.latest_block_height'

# Check attestation pool balance (after a few blocks)
threatattd query bank balances <attestation-module-address>
    --node http://localhost:26657
```

4. Create an attester key

```
threatattd keys add my-attester \
    --keyring-backend test \
    --home ~/.threatattd-local
```

The attester starts with **zero balance**. They need gas tokens to publish their first attestation.

5. Fund the attester (choose one)

Option A: Validator sends tokens

```
threatattestd tx bank send \  
  $(threatattestd keys show localkey -a --keyring-backend  
    test --home ~/.threatattestd-local) \  
  $(threatattestd keys show my-attester -a --keyring-backend  
    test --home ~/.threatattestd-local) \  
  1000000utatst \  
  --chain-id threatattest-local-1 \  
  --home ~/.threatattestd-local \  
  --keyring-backend test
```

Option B: Claim from the attestation incentive pool

Wait for the attestation pool to accumulate tokens (each block adds 3 TATST), then:

```
threatattestd tx attestation claim-reward \  
  --from my-attester \  
  --chain-id threatattest-local-1 \  
  --home ~/.threatattestd-local \  
  --keyring-backend test
```

Option C: Fee grant from validator

```
# Validator grants fee allowance to attester  
threatattestd tx feegrant grant \  
  $(threatattestd keys show localkey -a --keyring-backend  
    test --home ~/.threatattestd-local) \  
  $(threatattestd keys show my-attester -a --keyring-backend  
    test --home ~/.threatattestd-local) \  
  --spend-limit 1000000utatst \  
  --chain-id threatattest-local-1 \  
  --home ~/.threatattestd-local \  
  --keyring-backend test \  
  --from localkey
```

6. Publish first attestation

Once the attester has tokens, they can publish:

```
threatattestd tx attestation publish \  
  --artifact-type FILE \  
  --artifact-sha256 <sha256> \  
  --severity HIGH \  
  --confidence 80 \  
  --ttl 86400 \  
  --description "Sample attestation" \  
  --category TATST:MALWARE \  
  --from my-attester \  
  --chain-id threatattest-local-1 \  
  --home ~/.threatattestd-local \  
  --keyring-backend test
```

7. Monitor the attester's reputation

```
threatattestd query reputation reputation $(threatattestd keys  
  show my-attester -a --keyring-backend test --home  
  ~/.threatattestd-local)
```

Quick Start (all-in-one)

```
cd chain
```

```
# 1. Build  
make build
```

```
# 2. Init  
make localnet-init
```

```
# 3. Start  
make localnet-start
```

```
# 4. Create attester key  
./bin/threatattestd keys add demo-attester \  
  --keyring-backend test \  
  --home ~/.threatattestd-local
```

```
# 5. Fund attester (validator sends 1 TATST)
```

```

VALIDATOR=$(./bin/threatattestd keys show localkey -a --
keyring-backend test --home ~/.threatattestd-local)
ATTESTER=$(./bin/threatattestd keys show demo-attester -a --
keyring-backend test --home ~/.threatattestd-local)

./bin/threatattestd tx bank send "$VALIDATOR" "$ATTESTER"
1000000utatst \
--chain-id threatattest-local-1 --home ~/.threatattestd-
local --keyring-backend test -y

# 6. Publish first attestation
SHA=$(sha256sum /etc/hostname | cut -d' ' -f1)
./bin/threatattestd tx attestation publish \
--artifact-type FILE --artifact-sha256 "$SHA" \
--severity LOW --confidence 50 --ttl 3600 \
--description "Test attestation" --category TATST:MALWARE \
--from demo-attester --chain-id threatattest-local-1 \
--home ~/.threatattestd-local --keyring-backend test -y

# 7. Query it back
sleep 3
./bin/threatattestd query attestation is-malicious --artifact-
sha256 "$SHA" \
--node http://localhost:26657

```

Constraints

Item	Value
Minimum attestation	1,024 bytes
Gas for publish	~0.001 TATST
Pool claim rate	1% of pool per claim
Block reward	10 TATST (halves every ~4 years)
Validator genesis stake	100 TATST
Validator genesis balance	200 TATST

ThreatAttest on dewile.net

Deploy a ThreatAttest chain node and DNS-bound attester identity on dewile.net.

1. Overview

After deployment you will have:

- A **ThreatAttest chain node** running on `dewile.net` (or a subdomain)
- A **DNS-bound identity** proving `dewile.net` controls an attester key
- A **REST API** (LCD on port 1317) serving threat intelligence queries
- A **browser extension** configured to check downloads and URLs against your chain

Architecture

```
dewile.net
├─ DNS TXT records (identity proof)
│  └─ _tatkey.dewile.net      TXT  "v=TAT1 k=secp256k1
p=<pubkey> a=<addr>"
│  └─ _tatactive.dewile.net   TXT  "dewile"
│
├─ threatattestd (Cosmos SDK node)
│  └─ :26656  P2P
│  └─ :26657  RPC
│  └─ :1317   REST API (LCD) ← extension queries this
│  └─ :9090   gRPC
│
└─ Caddy / nginx (TLS termination for :1317 → :443)
```

2. Prerequisites

Requirement	Check
Go ≥ 1.23	<code>go version</code>
Docker ≥ 24 (optional)	<code>docker --version</code>
DNS control for <code>dewile.net</code>	Edit zone file or registrar panel
Ports 26656, 26657, 1317 open	Firewall / security group
TLS certificate for <code>api.dewile.net</code>	certbot / acme.sh

3. Build the Chain Binary

```
cd ~/work/ai/threat-attest/chain
make build
# → bin/threatattestd

# Add to PATH (or symlink, or use full path throughout)
export PATH="$PWD/bin:$PATH"
echo 'export PATH="$HOME/work/ai/threat-attest/chain/bin:
    $PATH"' >> ~/.bashrc
```

```
threatattestd version
```

4. Initialize the Node

```
export CHAIN_ID=dewile-1
export MONIKER=dewile-validator
export HOME_DIR=~/.threatattestd-dewile

# Init
threatattestd init $MONIKER \
    --chain-id $CHAIN_ID \
    --home $HOME_DIR

# Create validator key
threatattestd keys add validator \
    --keyring-backend file \
    --home $HOME_DIR

# Create attester key (this key will be DNS-bound to
    dewile.net)
threatattestd keys add dewile-attester \
    --keyring-backend file \
    --home $HOME_DIR

# — CRITICAL: Set keyring-backend default
# The Cosmos SDK default keyring backend is "os" (OS
    keychain).
# Keys created with "file" are invisible to "os". Set the
    default
# so every subsequent command doesn't need --keyring-backend
    file.
sed -i 's/keyring-backend = "os"/keyring-backend = "file"/' \
```

```
$HOME_DIR/config/client.toml

# Verify keys are accessible
threatattestd keys list --home $HOME_DIR
VALIDATOR_ADDR=$(threatattestd keys show validator -a --home
    $HOME_DIR)
ATTESTER_ADDR=$(threatattestd keys show dewile-attester -a --
    home $HOME_DIR)
echo "Validator: $VALIDATOR_ADDR"
echo "Attester: $ATTESTER_ADDR"
```

5. Configure Genesis

```
VALIDATOR_ADDR=$(threatattestd keys show validator -a --home
    $HOME_DIR)

# Add genesis account with initial tokens
threatattestd genesis add-genesis-account $VALIDATOR_ADDR
    10000000000utast \
    --home $HOME_DIR

# Generate gentx
threatattestd genesis gentx validator 5000000000utast \
    --chain-id $CHAIN_ID \
    --home $HOME_DIR \
    --keyring-backend file

# Collect gentx
threatattestd genesis collect-gentxs --home $HOME_DIR
```

6. Configure app.toml (REST API)

Edit `$HOME_DIR/config/app.toml`:

```
[api]
enable = true
address = "tcp://0.0.0.0:1317"
swagger = true

[api-cors]
enabled = true
allowed-origins = ["*"]
```

```
allowed-methods = ["GET", "POST", "OPTIONS"]
allowed-headers = ["*"]
```

Edit `$HOME_DIR/config/config.toml` for RPC:

```
[rpc]
laddr = "tcp://0.0.0.0:26657"
cors_allowed_origins = ["*"]

[p2p]
laddr = "tcp://0.0.0.0:26656"
```

7. DNS Identity Setup

7.1 Generate the TXT Record Value

```
ATTESTER_ADDR=$(threatattestd keys show dewile-attester -a \
  --keyring-backend file --home $HOME_DIR)

ATTESTER_PUBKEY=$(threatattestd keys show dewile-attester -p \
  --keyring-backend file --home $HOME_DIR | jq -r '.key')

echo "v=TAT1 k=secp256k1 p=$ATTESTER_PUBKEY a=$ATTESTER_ADDR"
```

7.2 Publish DNS TXT Records

Add these records to the `dewile.net` DNS zone:

```
; ThreatAttest identity – selector "dewile"
_tatkey.dewile.net. 3600 IN TXT "v=TAT1 k=secp256k1
p=<pubkey_hex> a=<tatst_addr>"
_tatactive.dewile.net. 3600 IN TXT "dewile"
```

Verify DNS propagation:

```
dig +short TXT _tatkey.dewile.net
dig +short TXT _tatactive.dewile.net
```

7.3 DNSSEC (Recommended for Expert Tier)

DNSSEC-signed zones unlock Tier 3 (Expert, 2.00× confidence multiplier). If `dewile.net` has DNSSEC enabled, include the DS, DNSKEY, and RRSIG records in the identity registration transaction.

8. Start the Node

```
threatattestd start \  
  --home $HOME_DIR \  
  --minimum-gas-prices 0.025utast \  
  --x-crisis-skip-assert-invariants
```

For production, run under systemd:

```
# /etc/systemd/system/threatattestd.service  
[Unit]  
Description=ThreatAttest Node – dewile.net  
After=network.target  
  
[Service]  
Type=simple  
User=threatattest  
ExecStart=/home/threatattest/bin/threatattestd start \  
  --home /home/threatattest/.threatattestd-dewile \  
  --minimum-gas-prices 0.025utast  
Restart=on-failure  
RestartSec=10  
LimitNOFILE=65536  
  
[Install]  
WantedBy=multi-user.target  
  
sudo systemctl daemon-reload  
sudo systemctl enable --now threatattestd  
sudo systemctl status threatattestd
```

Verify:

```
curl -s http://localhost:26657/status | jq  
  '.result.sync_info.latest_block_height'
```

9. TLS Termination for the REST API

The browser extension will query the REST API. For production, put TLS in front of port 1317.

Option A: Caddy (simplest)

```
# /etc/caddy/Caddyfile
api.dewile.net {
    reverse_proxy localhost:1317
}
```

Option B: nginx

```
server {
    listen 443 ssl;
    server_name api.dewile.net;

    ssl_certificate      /etc/letsencrypt/live/api.dewile.net/
fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.dewile.net/
privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:1317;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Verify:

```
curl -s https://api.dewile.net/cosmos/base/tendermint/v1beta1/
node_info | jq '.default_node_info.version'
```

10. Fund the Attester

```
VALIDATOR_ADDR=$(threatattestd keys show validator -a \
--keyring-backend file --home $HOME_DIR)
ATTESTER_ADDR=$(threatattestd keys show dewile-attester -a \
```

```
--keyring-backend file --home $HOME_DIR)

# Send 1,000 TATST to the attester
threattestd tx bank send $VALIDATOR_ADDR $ATTESTER_ADDR
10000000000utatst \
  --chain-id $CHAIN_ID \
  --home $HOME_DIR \
  --keyring-backend file \
  -y

# Verify
threattestd query bank balances $ATTESTER_ADDR \
  --node http://localhost:26657
```

To reach Tier 3 (Expert, 2.00x), the attester needs: - DNSSEC on `dewile.net` - 5,000+ reputation score - EXPERT flag set on the identity record

```
SHA=$(sha256sum /etc/hostname | cut -d' ' -f1)

threatattestd tx attestation publish \
  --artifact-type FILE \
  --artifact-sha256 $SHA \
  --severity LOW \
```

```
--confidence 50 \  
--ttl 3600 \  
--description "First attestation from dewile.net" \  
--category TATST:MALWARE \  
--attester-domain dewile.net \  
--attester-selector dewile \  
--from dewile-attester \  
--chain-id $CHAIN_ID \  
--home $HOME_DIR \  
--keyring-backend file \  
-y
```

Query it back:

```
threatattestd query attestation is-malicious \  
  --artifact-sha256 $SHA \  
  --node http://localhost:26657
```

13. Configure the Browser Extension

13.1 Build the Extension

```
cd ~/work/ai/threat-attest/extension  
make build-prod  
# → dist/
```

13.2 Set the Chain Endpoint

Before building (or in `dist/` after build), edit the API base URL:

```
// extension/src/api.js (or config)  
const CHAIN_API = "https://api.dewile.net";
```

Rebuild and load unpacked in `chrome://extensions`.

13.3 Verify

1. Navigate to any page. The toolbar badge should appear green.
2. Download a test file whose SHA-256 matches an active attestation.
3. The extension should block or warn based on severity.

14. Maintenance

Daily

```
# Check node health
curl -s https://api.dewile.net/cosmos/base/tendermint/v1beta1/
    node_info

# Check block sync
curl -s http://localhost:26657/status | jq '.result.sync_info'
```

Weekly

```
# Renew attestations near expiry
threatattestd query attestation list-by-attester
    $ATTESTER_ADDR --node http://localhost:26657

# Check reputation
threatattestd query reputation reputation $ATTESTER_ADDR --
    node http://localhost:26657

# Claim pool rewards
threatattestd tx attestation claim-reward \
    --from dewile-attester \
    --chain-id $CHAIN_ID \
    --home $HOME_DIR \
    --keyring-backend file -y
```

Identity Renewal

DNS identities expire after 1 year by default. Renew before expiry:

```
threatattestd tx identity renew-identity \
    --domain dewile.net \
    --selector dewile \
    --from dewile-attester \
    --chain-id $CHAIN_ID \
    --home $HOME_DIR \
    --keyring-backend file -y
```

15. Troubleshooting

Problem	Check
validator is not a valid name or address: decoding bech32 failed	Keyring backend mismatch. Keys created with <code>--keyring-backend file</code> but default is <code>os</code> . Fix: <code>sed -i 's/keyring-backend = "os"/keyring-backend = "file"/' \$HOME_DIR/config/client.toml</code>
keys show returns nothing	Run <code>threatattestd keys list --home \$HOME_DIR</code> to list keys and verify the correct backend
Node won't start	<code>journalctl -u threatattestd -n 50</code>
DNS identity stuck PENDING	DNS TXT records propagated? <code>dig TXT _tatkey.dewile.net</code>
Extension shows "Node offline"	TLS cert valid? <code>curl -v https://api.dewile.net/</code>
Attestation rejected	Reputation score $\geq$ minimum? Confidence in range 0-100? TTL between 1h and 1y?
Gas too low	<code>--minimum-gas-prices 0.025utatst</code> in <code>app.toml</code> or start flag

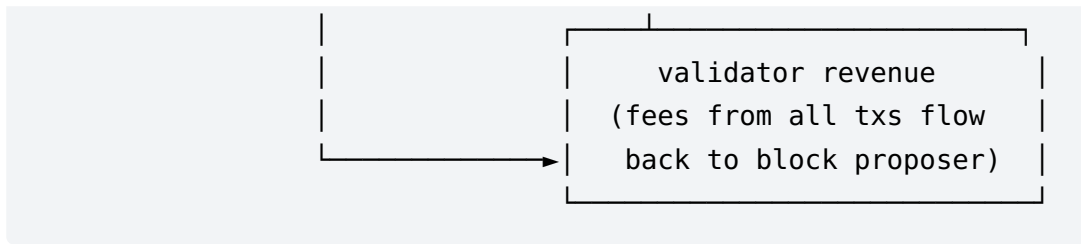
16. Next Steps After Setup

1. **Publish real attestations** — feed low-positive malware hashes from the plan10-web pipeline.
2. **Wire up YARA rules** — pin family-level rules to IPFS, reference CIDs in attestation `DetectionRules`.
3. **Add attesters** — invite other researchers to register identities and co-attest.
4. **Enable paid subscriptions** — `MsgSubscribe/MsgUnsubscribe` handlers are implemented; wire the subscription-tier rate limit into the query/API layer.
5. **List on Osmosis** — governance proposal for TATST token on the Cosmos DEX.

ThreatAttest Steady-State Flow

Token Circulation

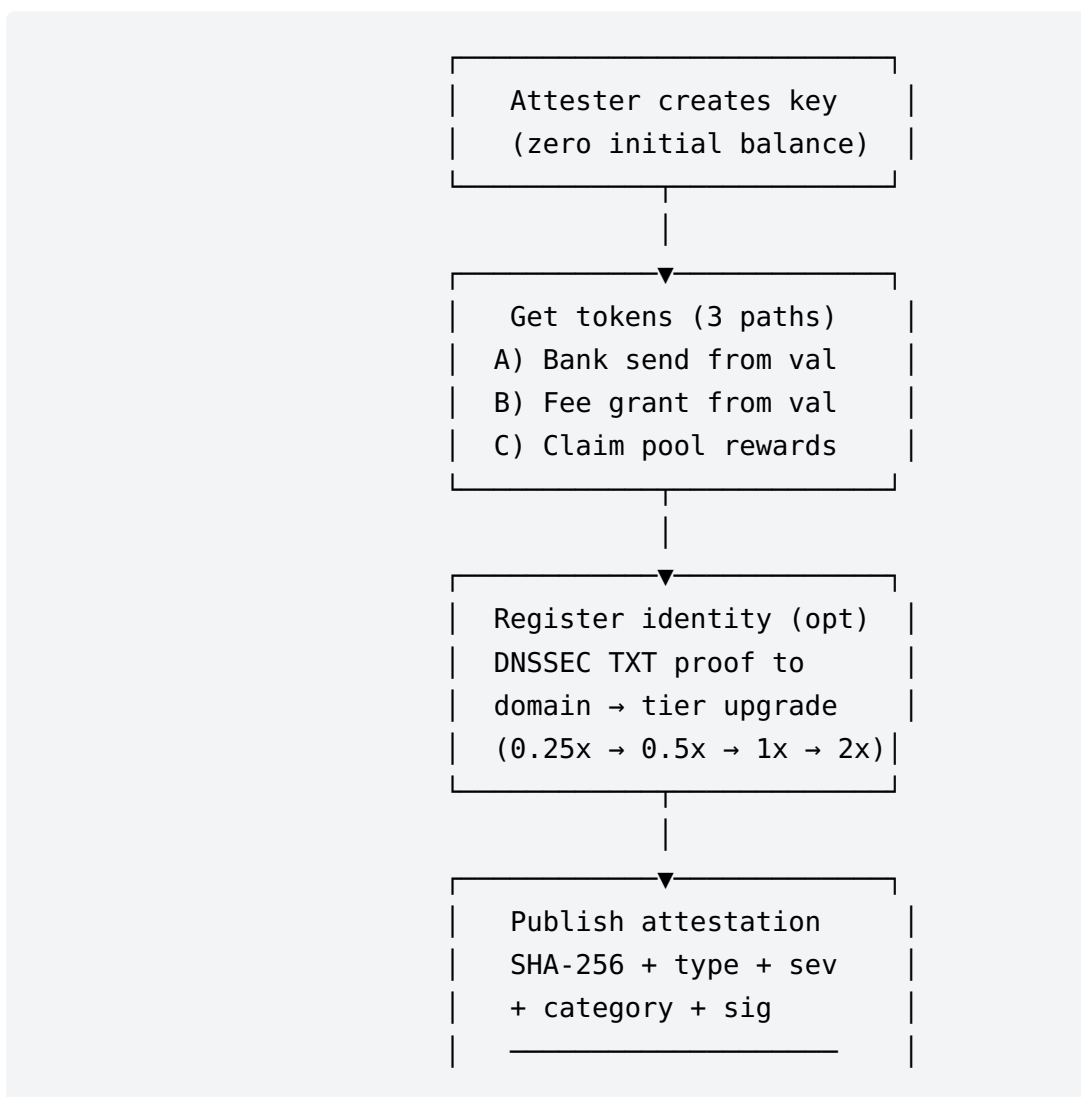
tatmint module

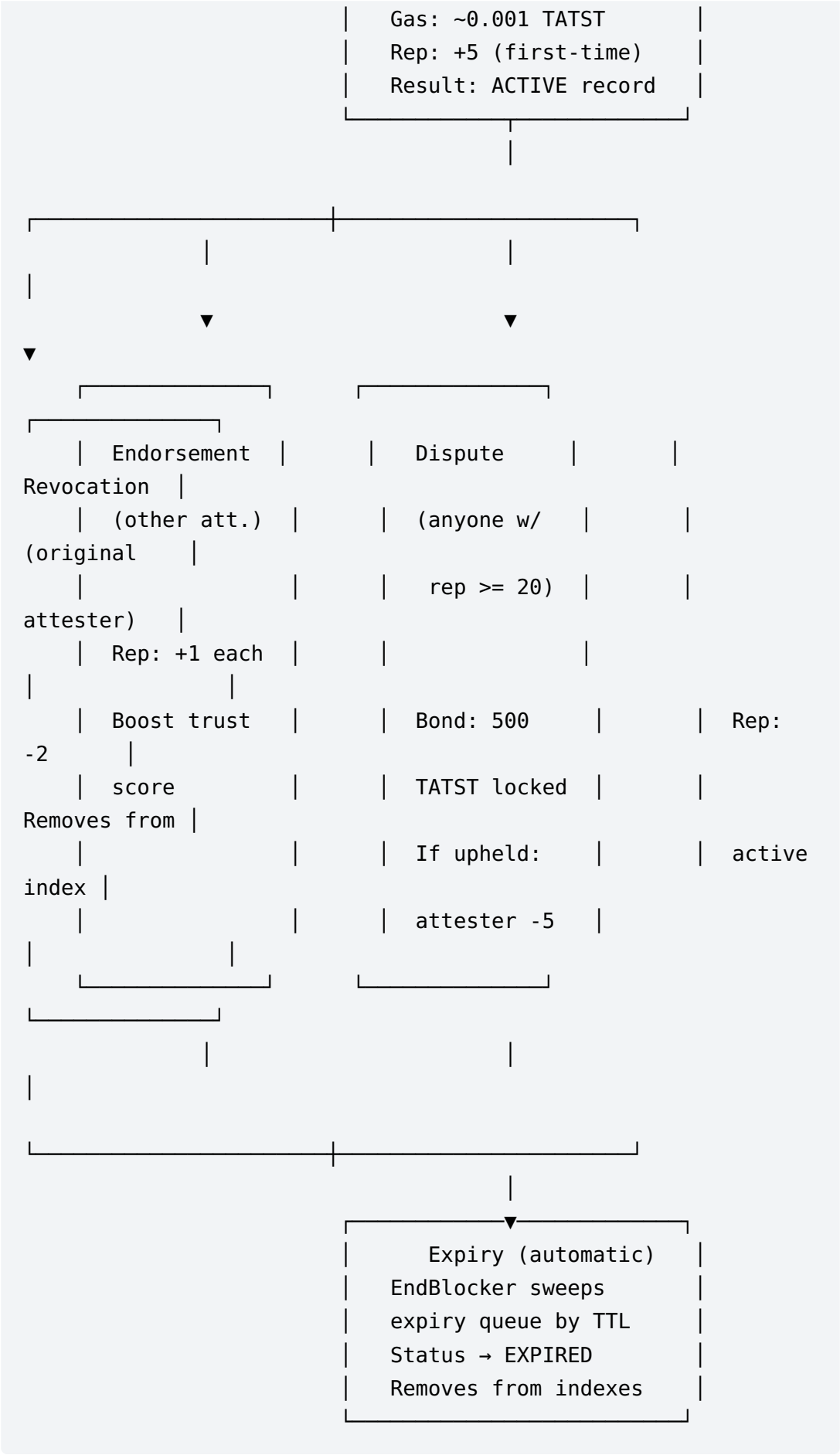


Key numbers per block (~2 seconds):

Metric	Value
Minted	10 TATST
Validator staking reward pool	5 TATST
Community pool	2 TATST
Attestation incentive pool	3 TATST
Gas for a publish tx	~0.001 TATST
Pool claim (1% after 100 blocks)	~3 TATST

Attestation Lifecycle





Attestation status transitions:

```
Publish → ACTIVE ─┬→ ENDORSED (count incremented)
                  │├→ DISPUTED (bond locked, remains in
active index)
                  │├→ REVOKED (only by original
attester)
                  │├→ EXPIRED (EndBlocker sweep, TTL
elapsed)
                  └┬→ SUPERSEDED (by newer attestation
for same artifact)
```

Identity & Reputation Flow

```
|
|                                     Identity
|
Registration                         |
|
|
|  1. Attester publishes TXT records to their DNS
zone:                               |
|      selector._tatkey.domain.com  TXT  "v=TAT1 k=secp256k1
p=..." |
|      selector._tatproof.domain.com TXT
"..." |
|      _tatactive.domain.com         TXT
"selector" |
|
|
|  2. Attester submits MsgRegisterIdentity with DNSSEC
evidence |
|      bundle (RRSIGs, DNSKEYs, DS records from parent
zone) |
|
|
|  3. Chain validates: freshness (<1h), algorithm (ECDSA
P-256), |
|      TXT record format, proof
signature |
|
|
|  4. Identity record created: status =
```

```
ACTIVE |
| Attester now has: DNS_BOUND tier (1.0x
confidence) |
| +200 reputation
score |
| Identity flags: EXPERT | ORGANIZATION | SECURITY_FIRM |
etc. |
```



```
| Tier
Determination |
|
|
|
GetTier(addr):
|
| if has ACTIVE identity AND flags&EXPERT AND score >=
5000: |
| → EXPERT (tier 3, 2.00x
confidence) |
| elif has ACTIVE
identity: |
| → DNS_BOUND (tier 2, 1.00x
confidence) |
| elif score >= staking threshold (delegation
proxy): |
| → STAKED (tier 1, 0.50x
confidence) |
|
else:
|
| → ANONYMOUS (tier 0, 0.25x
confidence) |
```



```
| Reputation
Scoring |
|
|
| Events that adjust
```

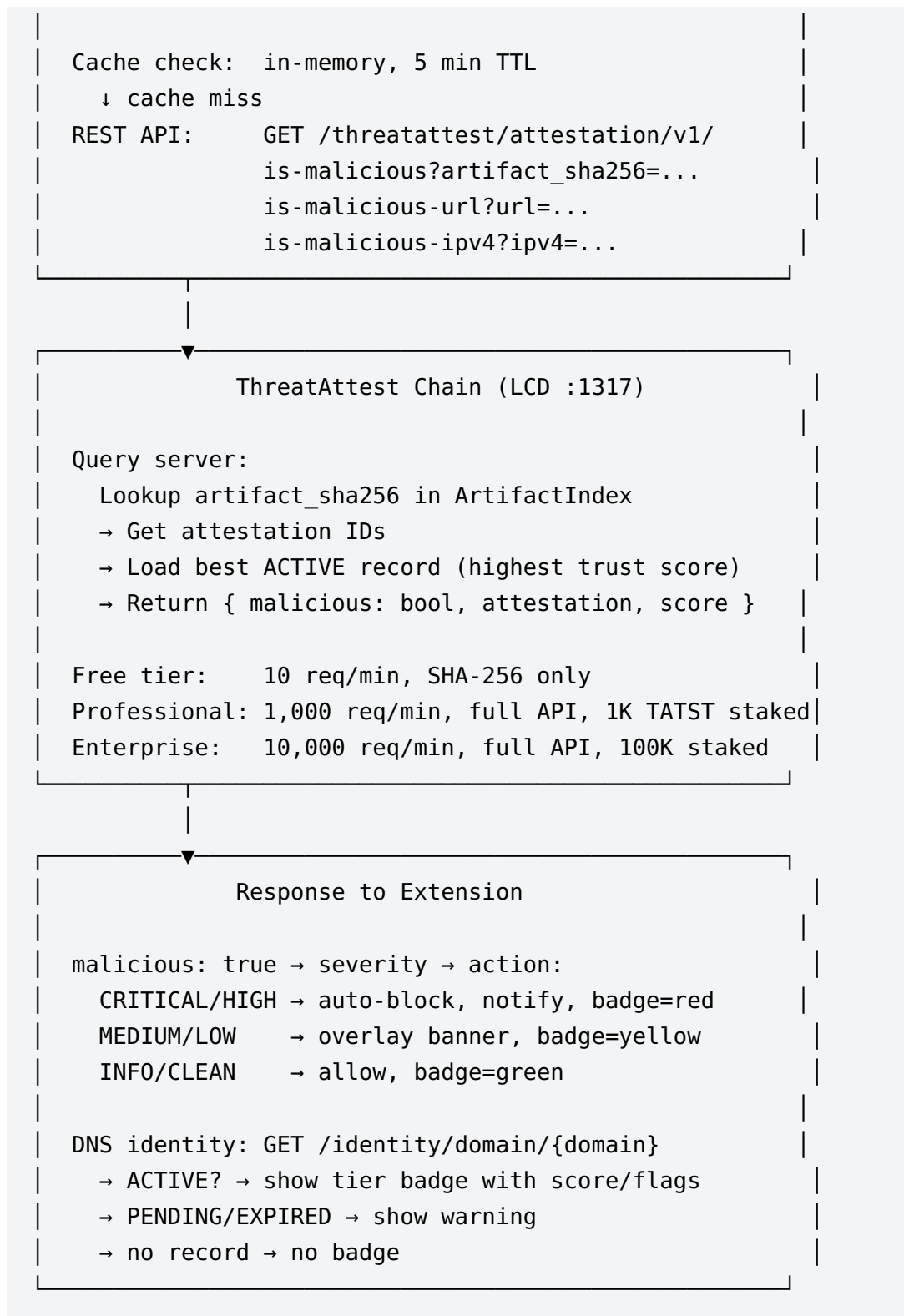
```

score:
|   Identity registration:      +200 (one-
time)
|   30-day milestone:
+50
|   Identity renewal:
+10
|   Publish attestation:      +5 (first-time
bonus)
|   Endorsement received:      +1 (to both attester and
endorser)
|   Valid dispute (upheld):    -5 (to
attester)
|   Revocation:                -2 (to
attester)
|   DNS record removed:        -200 (identity
penalty)
|   Voluntary revoke:          -100
(permanent)
|
|
|   Decay: 0.10%/day after 30 days of
inactivity
|   Range: 0 –
10,000
|   Blacklisted: score frozen, cannot publish, governance
removal

```

Query Flow (Browser Extension → Chain)





Additional Query Consumers

The browser extension is one of several consumers. Any system that encounters an IoC can query the chain:



```

| | Sentinel | | Blocky | | calling the
| |
| | | | REST API
| |
| | IoC enrichment | | Domain block | |
| |
| | pipeline | | at DNS level | |
| |
| |_____
|_____
|
|
| All consumers use the same REST API. No auth. No per-call
cost. |
| Rate limits gated by subscription tier (staked
TATST). |
|_____

```

Query vs Transaction Cost

Operation	Type	Cost	Where Set
is-malicious (SHA-256)	Query (GET)	Free	N/A — reads don't use gas
is-malicious-url (URL)	Query (GET)	Free	N/A
is-malicious-ipv4 (IP)	Query (GET)	Free	N/A
list-by-artifact	Query (GET)	Free	N/A
get-attestation	Query (GET)	Free	N/A
Node info / status	Query (GET)	Free	N/A
Publish attestation	Tx (POST)	~100K gas	root.go:1300 — MinGasPrices = "0.025utatst"
Endorse attestation	Tx (POST)	~80K gas	Same
Dispute attestation	Tx (POST)	~120K gas + 500 TATST bond	Same
Claim reward	Tx (POST)	~100K gas	Same
Subscribe (Professional)	Tx (POST)	~100K gas + 1,000 TATST stake	Same
Subscribe (Enterprise)	Tx (POST)	~100K gas + 100,000 TATST stake	Same

Transaction cost at current settings:

```
MinGasPrices = 0.025 utatst/gas-unit (root.go:1300)
```

```
Publish attestation: 100,000 gas × 0.025 = 2,500 utatst =  
0.0025 TATST
```

```
Endorse: 80,000 gas × 0.025 = 2,000 utatst =  
0.0020 TATST
```

```
Dispute (+bond): 120,000 gas × 0.025 = 3,000 utatst =  
0.0030 TATST + 500 TATST bond
```

```
Claim reward: 100,000 gas × 0.025 = 2,500 utatst =  
0.0025 TATST
```

Docker/localnet override: `entrypoint.sh` and `start-localnet.sh` set `--minimum-gas-prices 0utatst`, making **all transactions free** in development.

Why queries are free: Cosmos SDK separates reads (QueryServer, direct KV store access) from writes (MsgServer, mempool → consensus → state machine). Only writes pass through the gas meter. Rate limiting at the query layer is implemented via the subscription tiers (`GetSubscriptionTier`) rather than per-call charging — the token lock creates demand without making reads expensive or slow.

Setting the cost

The minimum gas price is set once at node startup in `cmd/threatattestd/cmd/root.go:1300`:

```
srvCfg.MinGasPrices = "0.025utatst"
```

This is a **validator-enforced minimum**, not a protocol parameter. Each validator can set their own minimum. Transactions offering below this price are rejected from the mempool. The value can be changed by editing the source and rebuilding, or setting `minimum-gas-prices` in `app.toml`.

The gas consumed per transaction type is estimated by the Cosmos SDK based on message size, signature verification, and state reads/writes. It is not directly configurable per message type — it derives from the operations the handler performs.

Governance Flow

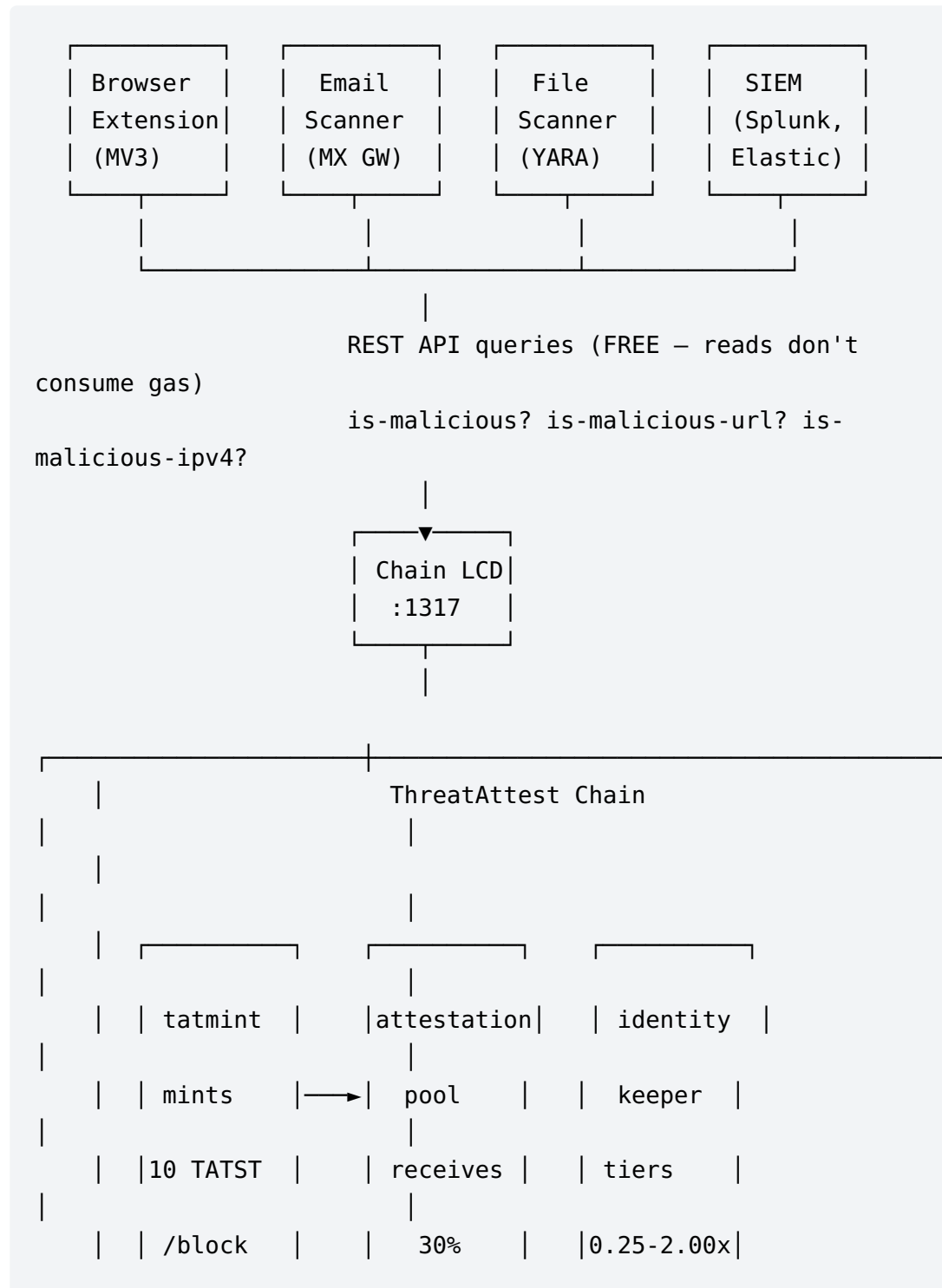


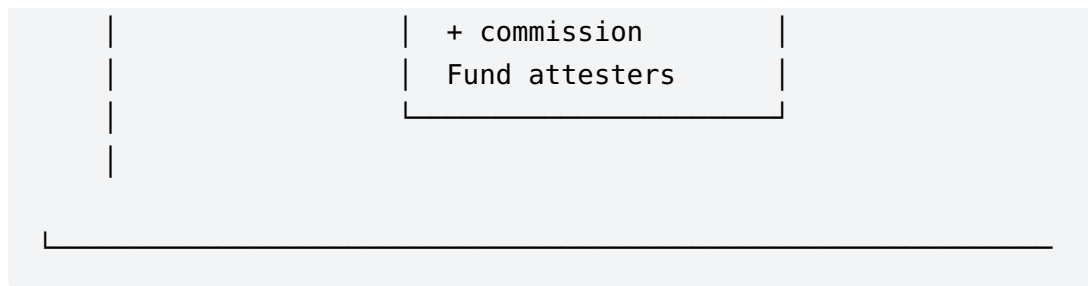
```

staking_threshold      |
|      reputation:    decay_rate,
max_gain_per_epoch    |
|      tatmint:      validator_reward_bps,
attester_reward_bps   |
|_____

```

Complete Steady-State (All Flows Combined)





Token lifecycle per block:

Mint 10 TATST

├─ 5.0 → fee_collector → distribution → validator reward pool

└─ ↓
validator claims → liquid TATST

└─ ↓
validator optionally sends or fee-grants to attesters

├─ 2.0 → community_pool (governance-controlled)

└─ 3.0 → attestation module account

└─ ↓
attester calls claim-reward

└─ ↓
attester spends gas to publish/endorse/dispute

└─ ↓
gas fees → validator (block proposer)

└─ ↓
cycle repeats

Reputation feedback loop:

Publish quality attestations → earn reputation → unlock higher tier →

higher confidence multiplier → more endorsements → more reputation →

claim more pool rewards → fund more attestations → repeat

Publish false attestations → disputed → reputation slashed → lower tier → bond locked → potential blacklist → exit

Closing the Loop: How a Threat Intel Token Goes From Gas to Value

2026-06-19 · 8 min read

Every Cosmos SDK chain starts the same way: empty genesis, zero balances, a validator with a self-delegation, and a freshly minted token that nobody wants. The real work isn't getting the chain to produce blocks — it's getting someone to *care* about the token.

ThreatAttest launched with a clean thesis: **pay people to publish verifiable threat intelligence**. The block reward splits 50% to validators, 20% to the community pool, and 30% to an attestation incentive pool. Attesters earn tokens for quality submissions. Publishers bear reputation risk. The chain produces a cryptographically-auditable feed of threat data.

That's the supply side. But a token with only supply and no demand is a loyalty program with no store. Here's the strategy to close the loop.

The Problem: Circular Value

Today the chain works like this:

Block reward → 30% → attestation pool → attester claims →
attester spends gas → validators earn fees

|



It's a closed circuit. Tokens circulate between attesters and validators. Reputation scores go up. Attestations accumulate. But nobody *outside* the system has a reason to hold TATST. The token has utility but no external demand.

For TATST to have real value — value that makes a security researcher's attestation work worth real money — someone needs to **buy in from the outside**.

The Strategy: Four Tiers

Tier 1: Consume-to-Access (Immediate)

The simplest source of demand: make the threat feed *useful* and charge for access.

The REST API is currently free and unauthenticated. That works for a devnet. For production, it should look like this:

Tier	Rate limit	Stake required
Free	10 req/min, SHA-256 only	None
Professional	1000 req/min, URL + IPv4	1,000 TATST locked
Enterprise	10,000 req/min, full API	100,000 TATST locked

The stake isn't consumed — it's locked in a module account and returned when the subscription ends. While locked, it creates scarcity in the circulating supply. A security vendor running an enterprise deployment of ThreatAttest Shield across 50,000 endpoints would need to lock 100,000 TATST. That's 100,000 tokens pulled from the market.

The chain also takes a small burn on each query (e.g., 1 utatst), adding constant deflationary pressure proportional to usage.

How to build it: Add a `MsgSubscribe` handler to the attestation module that locks tokens and sets a rate-limit record in the KV store. An ante handler check reads the record and allows the query through. Token lock periods of 30/90/365 days.

Tier 2: Attestation-as-a-Service (Medium-term)

Security companies don't just want a feed — they want *specific* intelligence. Phishing domains. C2 infrastructure. Malware hashes. The kind of intelligence that saves their customers from breaches.

This creates a natural marketplace:

```
CrowdStrike wants early warning on new Cobalt Strike C2
infrastructure
  → Posts a bounty: 50,000 TATST for verified C2 attestations
  (DOMAIN type, TATST:C2 category)
```

Independent researchers see the bounty

- Deploy honeypots, extract C2 domains
- Publish attestations with DNSSEC-anchored identities
- Claim bounty on verification

CrowdStrike gets real-time intel, researcher gets paid, chain earns fees

A `MsgCommissionAttestation` creates a time-bounded bounty with locked funds. Attesters publish attestations referencing the commission ID. The commissioner marks one as accepted, releasing the bounty to the attester. If unclaimed by deadline, the bounty returns to the commissioner.

How to build it: Add commission CRUD to the attestation keeper, a `MsgCommissionAttestation` and `MsgClaimCommission` pair, and an `EndBlocker` that refunds expired commissions.

Tier 3: Reputation Staking (Medium-term)

Chain-of-trust matters in threat intelligence. An anonymous Twitter account claiming “Emotet C2 at 192.168.1.1” is noise. A DNSSEC-bound identity from `security-firm.com` with a 5,000 reputation score claiming the same thing is a signal worth acting on.

The identity module already defines four tiers:

Tier	Name	Confidence multiplier	Requirements
0	Anonymous	0.25×	Nothing
1	Staked	0.50×	500 TATST delegated
2	DNS Bound	1.00×	Active DNSSEC identity
3	Expert	2.00×	DNS identity + 5,000 reputation + EXPERT flag

Staking tokens for higher tiers creates demand. An expert attester needs 500+ TATST staked *and* an established reputation.

Organizations running multiple identities (SOC team, threat research, automated honeypots) each need their own stake.

The stake also serves as **economic security**: false attestations can be disputed. A successful dispute — say, filing a fabricated IoC — slashes the attester’s reputation and burns a portion of their

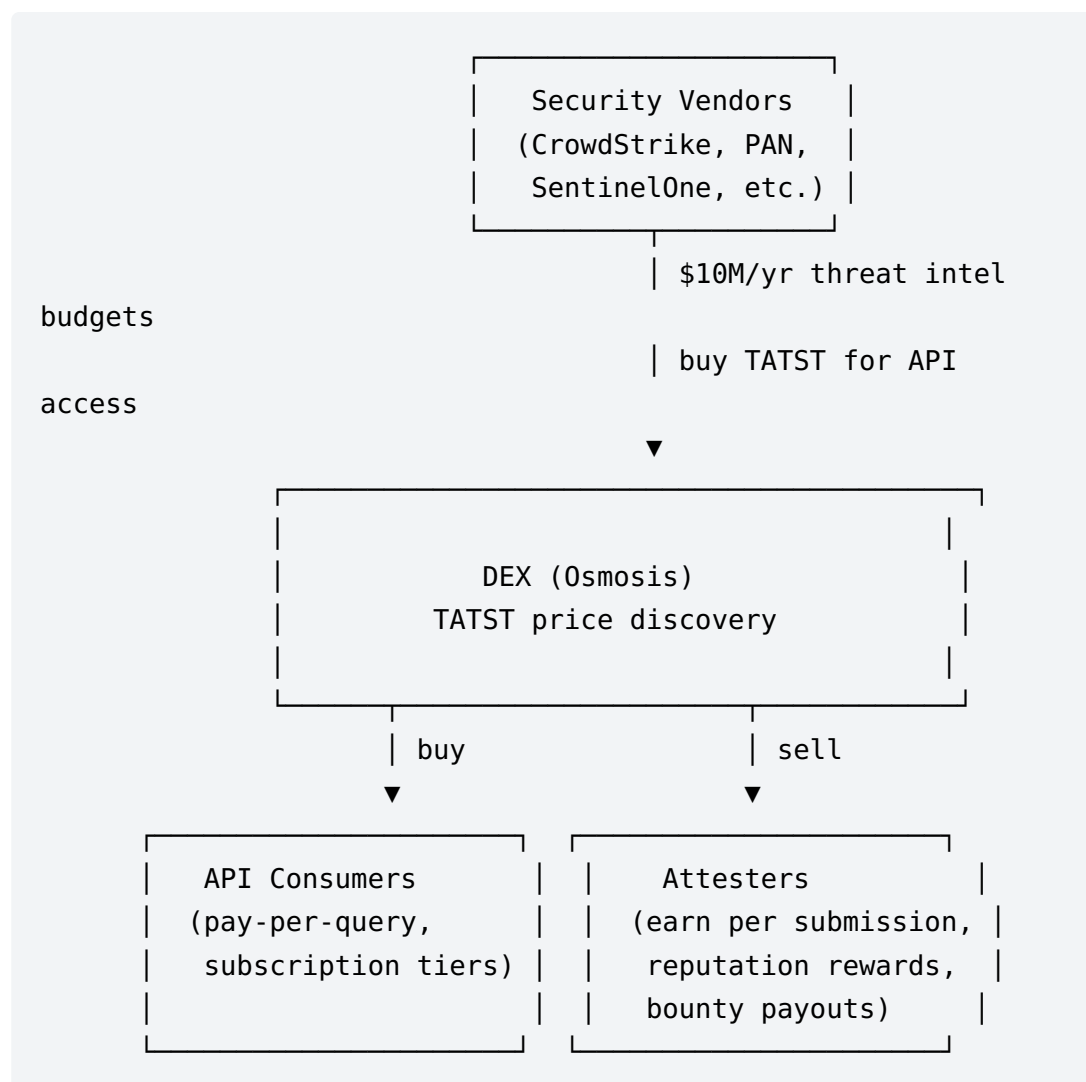
stake. The dispute bond (500 TATST) creates an anti-spam mechanism. False disputes lose the bond; valid disputes earn reputation.

This is a **prediction-market dynamic** in disguise: attesters stake on their accuracy, disputers stake on their inaccuracy, and the chain resolves the outcome through reputation-weighted community consensus.

How to build it: The identity, reputation, and dispute infrastructure already exists in the chain. Enable identity module registration (fix SDK v0.50 wire-up), connect dispute resolution to stake slashing, and add governance parameters for slash fractions.

Tier 4: The Decentralized Threat Marketplace (Long-term)

The end state: ThreatAttest is a **self-sustaining threat intelligence economy**.



The token velocity equation:

$$\text{Token market cap} \approx (\text{annual consumer spend}) / (\text{annual token velocity})$$

If security vendors spend \$10M/year on TATST for API access, and tokens circulate 4 times per year (locked stake, multi-month subscriptions, long-term commissions), the market cap floor is approximately \$2.5M. At 1% adoption of the \$10B enterprise threat intelligence market, that's \$100M in annual spend and a \$25M market cap — not because of speculation, but because the token *does useful work*.

What To Build First

If you're contributing to ThreatAttest today, here's the implementation priority:

#	Feature	Effort	Impact
1	Paid API subscriptions — MsgSubscribe + rate-limit ante handler	~1 week	Creates immediate token demand
2	Reputation staking/slashing — wire identity, connect disputes to slash	~1 week	Economic security, stake demand
3	Attestation commissions — bounty creation, claiming, expiry sweep	~2 weeks	Incentivizes specific intelligence
4	DEX listing — TATST on Osmosis via governance proposal	~1 week	Price discovery, external buyers
5	Enterprise contracts — direct fiat→token conversions for vendors	~3 months	Real external capital inflow

Steps 1–3 are pure chain development. Steps 4–5 are ecosystem and business development. All five are necessary for the loop to close.

The Unsexy Truth

Blockchains don't create value. They transfer it.

A block reward is a subsidy. A DEX listing is a venue. A token price is a signal.

None of these matter unless there's a real economic actor on the other side who **needs the thing the chain produces** and is willing to pay for it.

ThreatAttest's bet is that the thing — a decentralized, reputation-weighted, cryptographically-verifiable threat intelligence feed — is worth paying for. If that bet is right, the token captures the value automatically. If it's wrong, no amount of tokenomics can fix it.

The strategy above is the plan for testing the bet. Start with API access. Prove someone will pay for the data. Build from there.

ThreatAttest is an open-source project. The chain source is at github.com/threatattest/chain. The browser extension is ThreatAttest Shield, available in the same repository.
